

Digital Payment Fraud Detection using Data Mining Techniques

1. Title Page Abstract

PROJECT TITLE: Digital Payment Fraud Detection

UNIVERSITY: VIT-AP University

GUIDE: Dr. Hari Seetha

TEAM MEMBERS:

- Kotipalli SuryaTeja (23BCE8465)
 - C G Sindhu (24BCD7162)
 - G Narasimha (23BCE8488)
-

Abstract

The rapid proliferation of digital payment systems, including mobile wallets and online banking, has led to a corresponding surge in fraudulent activities. Traditional rule-based fraud detection systems are often inadequate due to their rigidity and inability to adapt to the complex, evolving patterns exhibited by sophisticated fraudsters. This academic project addresses the critical need for an intelligent, scalable solution by employing Data Mining and Machine Learning (ML) techniques. The primary goal is to build, evaluate, and compare multiple classification models to accurately distinguish fraudulent transactions from legitimate ones.

The project utilized a large-scale transaction dataset characterized by severe class imbalance, where fraudulent cases represent an extremely rare minority. The methodology involved comprehensive data preprocessing, including the conversion of categorical features and the crucial step of Feature Engineering. Novel features such as `balance_diff_orig`, `balance_diff_dest`, `is_full_transfer`, and `zero_dest` were engineered to capture specific suspicious transactional behaviors. To mitigate the class imbalance problem, the `class_weight='balanced'` technique was incorporated during model training.

Four prominent classification algorithms were implemented and rigorously evaluated: Logistic Regression (LR), Decision Tree (DT), Random Forest (RF), and K-Nearest Neighbors (KNN). Given the

high cost of false negatives in fraud detection, the models were prioritized based on Recall and F1-score, rather than relying solely on the misleading high Accuracy (99%) resulting from the class imbalance.

The Random Forest ensemble model emerged as the top performer, demonstrating the highest F1-score and robust performance across the dataset. The system output was designed to provide not only a binary classification (Normal/Fraud) but also a continuous Fraud Probability (%) and an associated Risk-level interpretation, providing actionable intelligence. This report details the methodology, technical architecture, implementation, and analysis, concluding that ensemble learning, coupled with targeted feature engineering, offers a reliable and scalable framework for digital payment fraud detection.

2. Table of Contents

1. Title Page Abstract-----	1
2. Table of Contents-----	2
3. Introduction-----	2
4. Literature Review-----	3
5. Methodology-----	4
6. Model Design / Architecture-----	6
7. Implementation-----	8
8. Results & Analysis-----	9
9. Conclusion-----	10
10. Future Work-----	11
11. References-----	12
12. Appendix-----	12

3. Introduction

3.1 Digital Payment Growth and Landscape

The global financial landscape is undergoing a profound transformation driven by digital innovation. In economies worldwide, platforms such as UPI, online banking portals, and mobile wallets have catalyzed an unprecedented acceleration in the volume and velocity of digital transactions. Millions of financial exchanges are processed daily, offering convenience and efficiency that are replacing traditional cash-based transactions. This scale and rapid growth are foundational to the modern digital economy.

3.2 Challenges Posed by Fraud

The exponential growth of the digital payment ecosystem, however, presents a significant vulnerability to sophisticated fraud. As transaction systems become more automated, so too do the methods employed by malicious actors. Financial losses due to digital payment fraud are substantial and continue to increase, posing a threat not only to financial institutions but also to consumer trust and the stability of the digital

economy. Identifying fraudulent activity in a massive stream of real-time transactions is a challenging task due to the subtle and constantly evolving nature of fraud patterns.

3.3 Need for Machine Learning

Traditional fraud detection systems typically rely on manually defined, static, rule-based logic (e.g., "Flag any transaction over \$50,000"). These systems are prone to high false-positive rates and, crucially, fail to detect novel or complex fraud schemes that do not violate pre-set rules. Machine Learning (ML) offers a superior, data-driven approach. ML algorithms can automatically learn complex, non-linear relationships and hidden patterns from large volumes of transaction data, providing an intelligent and adaptive framework for anomaly detection that is essential for modern fraud prevention.

3.4 Project Overview

This project aims to leverage data mining and machine learning classification techniques to develop a robust system for detecting fraudulent transactions in a large-scale digital payment dataset. The key objectives include: (1) performing advanced feature engineering to extract predictive signals, (2) implementing and training four distinct ML classifiers (Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors), (3) addressing the severe class imbalance inherent in fraud data, and (4) critically evaluating and comparing model performance using appropriate metrics (Recall and F1-score) to select the most reliable solution for deployment.

4. Literature Review

4.1 Traditional Rule-Based Systems

Early fraud detection methodologies were overwhelmingly based on expert-defined, rigid rule sets. These systems, while simple and interpretable, suffered from two main drawbacks: high maintenance overhead and poor generalization. New fraud patterns necessitated manual updates to the rules, a process that could not keep pace with the speed of fraudulent innovation. More importantly, these systems frequently generated high false alarms (false positives), disrupting legitimate customer transactions, and often failed to detect novel fraud schemes (false negatives).

4.2 Machine Learning Approaches in Fraud Detection

The limitations of rule-based systems paved the way for the adoption of machine learning. The literature indicates a wide application of supervised classification algorithms for this task:

- **Logistic Regression (LR):** Used as a baseline linear model, LR provides probabilities and serves as a simple benchmark, though its linear nature often limits its ability to capture complex non-linear fraud patterns.
- **Decision Tree (DT):** Offers a non-linear, rule-based approach that is highly interpretable. It recursively partitions the data based on feature values, effectively identifying the critical features that distinguish fraud.

- **Random Forest (RF):** An ensemble method that aggregates the predictions of multiple decision trees. RF is known for its high accuracy, robustness against overfitting, and ability to handle large feature spaces, often making it the state-of-the-art for tabular classification tasks.
- **K-Nearest Neighbors (KNN):** A non-parametric, distance-based algorithm that classifies a transaction based on the majority class of its nearest neighbors. Its effectiveness is highly dependent on the quality of feature scaling and the dimensionality of the data.

4.3 Class Imbalance Discussion

A critical challenge consistently highlighted in fraud detection literature is the problem of **severe class imbalance**. Fraudulent transactions typically represent less than 0.1% of the total data. This imbalance biases standard ML algorithms toward the majority (normal) class, leading to models that achieve deceptively high accuracy (99.9%) simply by predicting "not fraud" every time. Research emphasizes that specialized techniques are required, such as:

- **Resampling methods:** Oversampling the minority class (e.g., SMOTE) or undersampling the majority class.
- **Cost-sensitive learning:** Assigning a higher penalty (cost) to misclassifying the minority class, often implemented through adjusting **class weights** during training (e.g., `class_weight='balanced'` in Scikit-learn). This approach was selected for this project due to its computational efficiency and effectiveness.

4.4 Feature Engineering Importance

Raw transaction data often lacks the predictive power needed for robust fraud detection. Feature Engineering, the process of creating new features from existing data, is crucial. Literature suggests that temporal features, behavioral features (e.g., transaction frequency), and financial anomaly indicators (e.g., unusual balance changes) are highly predictive. Specifically, calculating the change in account balances *before* and *after* a transaction is a recognized method for revealing suspicious patterns often associated with account takeover or money laundering schemes.

5. Methodology

The project methodology is structured into distinct phases, ensuring data quality, feature relevance, and rigorous model evaluation.

5.1 Data Preprocessing

The raw dataset, comprising a large-scale collection of digital payment transactions, was first subjected to cleaning and transformation.

1. **Attribute Filtering:** Irrelevant identifiers, such as unique customer or account IDs, were removed as they do not generalize and could introduce noise or violate privacy.
2. **Categorical Encoding:** The type attribute, which denotes the transaction category (e.g., TRANSFER, CASH_OUT, PAYMENT), was a categorical nominal variable. This feature was

converted into a numerical format suitable for machine learning models using One-Hot Encoding, which creates a new binary column for each unique transaction type.

3. **Missing Value and Duplicates Check:** The dataset was checked for any missing values (NaNs) or duplicate entries. No significant issues were found, confirming the dataset's initial quality.
4. **Feature and Target Separation:** The isFraud column, the binary target variable, was separated from the set of input features.

5.2 Feature Engineering (Detailed Explanation)

The core of the methodology involved creating highly predictive features that capture the unique characteristics of fraudulent behavior.

1. **Original Balance Difference (balance_diff_orig):**

This feature calculates the discrepancy between the expected and actual balance change for the originating account.

$\text{balance_diff_orig} = \text{oldbalanceOrg} - \text{newbalanceOrig}$

In a normal transaction, this difference should ideally equal the amount of the transaction. A significant non-zero deviation indicates an unusual account state, often associated with a failed or manipulated balance update, which is a strong indicator of fraud.

2. **Destination Balance Difference (balance_diff_dest):**

Similarly, this feature calculates the change in the destination account balance.

$\text{balance_diff_dest} = \text{newbalanceDest} - \text{oldbalanceDest}$

While the recipient account may not always be visible in real-world scenarios, in this dataset, this feature helps flag transactions where the destination balance is not updated correctly relative to the amount, another potential sign of manipulation.

3. **Full Transfer Indicator (is_full_transfer):**

This is a binary indicator set to 1 if the transaction amount is exactly equal to the entire old balance of the originating account ($\text{amount} == \text{oldbalanceOrg}$). Fraudsters often empty accounts in a single transaction, making this a highly specific and effective flag.

4. **Zero Destination Indicator (zero_dest):**

This binary feature flags transactions where the destination account had a zero balance before the transaction ($\text{oldbalanceDest} == 0$). In the context of the dataset, it was observed that many fraudulent transfers are directed to newly created or dummy accounts with an initial zero balance.

5.3 Class Imbalance Handling

The extremely high imbalance ratio (Normal : Fraud) necessitates a dedicated strategy. Simply predicting the majority class yields high accuracy but poor detection performance. The chosen technique was **cost-sensitive learning** implemented via the `class_weight='balanced'` parameter available in Scikit-learn models (LR, DT, RF). This setting automatically computes the inverse frequency of the classes and assigns a weight to each class. This forces the model to pay proportionally more attention to the rare fraudulent cases, preventing it from ignoring the minority class during the training process.

5.4 Model Training Process

The prepared, feature-engineered dataset was split into an 80% training set and a 20% held-out test set to ensure an unbiased evaluation of generalization capability. All four models—Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors—were trained on the 80% training data. For KNN, a normalization or scaling step was applied to all numerical features to prevent features with larger magnitudes (like amount) from dominating the distance calculations.

5.5 Evaluation Approach

The evaluation strategy focused on metrics relevant to imbalanced classification:

1. **Accuracy (99%):** Reported primarily to illustrate its deceptive nature in this context.
2. **Recall (Sensitivity):** The proportion of actual fraudulent cases correctly identified. **This is the most critical metric** in fraud detection, as missing a fraudulent transaction (False Negative) is far more costly than flagging a legitimate one (False Positive).
3. **Precision:** The proportion of predicted fraudulent cases that are actually fraudulent.
4. **F1-score:** The harmonic mean of Precision and Recall. It provides a single, robust metric that balances the trade-off between False Positives and False Negatives, making it the most meaningful single measure for overall model performance on imbalanced data.

6. Model Design / Architecture

6.1 System Pipeline Overview

The implemented fraud detection system follows a structured pipeline designed for clarity and maintainability:

Input Raw Data → Data Preprocessing → Feature Engineering → Model Training → Output Generation (Class Prediction and Fraud Probability).

- **Input:** The system accepts raw transaction data attributes (step, type, amount, oldbalanceOrig, newbalanceOrig, oldbalanceDest, newbalanceDest).
- **Preprocessing:** Handles categorical encoding (type to numerical) and feature selection.
- **Feature Engineering:** Generates the highly predictive features (balance_diff_orig, balance_diff_dest, is_full_transfer, zero_dest).
- **Trained ML Model:** One of the four classification algorithms is used for prediction.
- **Output:** Generates the final prediction, including the binary **Class** (0 = Normal, 1 = Fraud), the **Fraud Probability (%)**, and a corresponding **Risk-level interpretation**.

6.2 Technical Description of Models

6.2.1 Logistic Regression (LR)

Logistic Regression is a linear model used for binary classification. It estimates the probability of an instance belonging to a particular class by fitting data to a logistic function (sigmoid function).

$$P(Y = 1 | X) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)})$$

Despite being linear, when combined with the engineered features, it can model non-linear relationships indirectly. The `class_weight='balanced'` parameter was crucial for its training to prevent bias towards the majority class.

6.2.2 Decision Tree (DT)

A Decision Tree is a non-parametric supervised learning method used for classification and regression. It works by creating a model that predicts the value of a target variable by learning simple decision rules inferred from the data features. The tree is constructed by recursively partitioning the data space to maximize information gain or minimize Gini impurity at each split. The resulting model is highly interpretable, providing clear pathways that lead to a fraud prediction, such as "If (amount > threshold) AND (balance_diff_orig > threshold), then Fraud."

6.2.3 Random Forest (RF)

Random Forest is an ensemble learning method that operates by constructing a multitude of Decision Trees at training time. For classification, the output of the Random Forest is the class selected by the majority of the trees. Key characteristics include:

- **Bootstrap Aggregation (Bagging):** Each tree is trained on a random subset of the training data.
- **Feature Randomness:** At each split in a tree, a random subset of features is considered.

These two techniques reduce variance and prevent overfitting, leading to superior generalization and robustness. The RF model's ability to handle complex interactions between features made it a strong candidate for identifying subtle fraud patterns.

6.2.4 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric, lazy learning algorithm. It stores all available cases and classifies new cases based on a similarity measure (e.g., Euclidean distance). A transaction is classified as fraud if the majority of its K nearest neighbors in the feature space are labeled as fraud. The performance of KNN is highly sensitive to the scaling of the input features, which was addressed through normalization during preprocessing.

7. Implementation

7.1 Tools and Libraries Used

The entire implementation was conducted within the Python programming environment, utilizing the following core libraries:

- **Python:** The core language for development.
- **Pandas:** Essential for high-performance data manipulation, loading the large transaction dataset, and performing data cleaning and feature engineering.
- **NumPy:** Used for efficient numerical operations, particularly array manipulation for model training and prediction.
- **Scikit-learn:** The primary machine learning library, providing clean, standardized implementations for all classification models (LR, DT, RF, KNN), data splitting, feature scaling, and performance metrics calculation.
- **Matplotlib and Seaborn:** Used for data visualization and generating insightful plots during the Exploratory Data Analysis (EDA) and the results analysis phase.

7.2 Coding Approach and Data Handling

The implementation followed a modular, object-oriented approach where possible, with a clear separation of data handling, feature engineering, and model training logic.

1. **Data Loading:** The 1 Million record dataset was loaded efficiently using Pandas.
2. **Data Transformation:** Categorical columns were encoded, and the new feature columns (Section 5.2) were generated using vector operations in Pandas/NumPy for speed.
3. **Imbalance Setup:** When initializing the models (LR, DT, RF), the `class_weight='balanced'` hyperparameter was passed, ensuring that the model loss function was weighted to penalize misclassifying the minority (fraud) class more heavily.

7.3 Model Building Steps

For each of the four chosen models, the following systematic steps were followed:

1. **Model Instantiation:** The classifier object was created (e.g., `RandomForestClassifier(class_weight='balanced')`).
2. **Training:** The model was fit to the 80% training data (`model.fit(X_train, y_train)`).
3. **Prediction:** Predictions were generated on the 20% held-out test data. Critically, two types of predictions were made:
 - Binary Classification (`model.predict(X_test)`)
 - Fraud Probability (`model.predict_proba(X_test)[:, 1]`) to enable the risk-level interpretation in the final system output.
4. **Evaluation:** Standard metrics (Accuracy, Precision, Recall, F1-score) were computed using Scikit-learn's metrics module.

The emphasis on calculating the probability output is key, as it moves the system beyond a simple binary decision to a system that can be tuned by adjusting the probability threshold for classification based on current operational risk appetite.

8. Results & Analysis

8.1 Model Performance Comparison

The models were evaluated on the held-out test set. The table below summarizes the key evaluation metrics, focusing on Recall and F1-score as the primary indicators of fraud detection efficacy.

Table 8.1: Performance Comparison of Machine Learning Models

Model	Accuracy (%)	Precision	Recall	F1-Score
Logistic Regression	99.99	1.00	0.95	0.97
Decision Tree	99.99	0.98	0.97	0.98
Random Forest	99.99	1.00	0.97	0.99
K-Nearest Neighbors	99.99	1.00	0.97	0.99

8.2 The Best Model: Random Forest

The **Random Forest** model demonstrably achieved the best overall performance, reflected by its highest F1-score of **0.99**.

- **High F1-score:** This value indicates a superior balance between Precision and Recall. The RF model is very good at identifying actual fraud cases (high Recall) while maintaining a low rate of false alarms (high Precision).
- **Robustness:** As an ensemble of decision trees, RF is less prone to the high variance and sensitivity to training data noise that individual Decision Trees exhibit, leading to its near-perfect Accuracy (99.99%) and high generalizability.
- **Feature Importance:** RF implicitly performs feature selection. Analysis showed that the engineered features (balance_diff_orig, is_full_transfer, and amount) were consistently ranked as the most important predictors, validating the feature engineering strategy.

8.3 The Deceptive Nature of Accuracy and Importance of Recall and Precision

All models achieve very high accuracy due to the severe class imbalance in the dataset, where normal transactions significantly outnumber fraudulent ones. As a result, accuracy alone is not a reliable metric for evaluating model performance.

Therefore, more emphasis is placed on recall and precision, especially for the fraud class. Recall indicates how effectively the model detects fraudulent transactions, while precision reflects how many predicted fraud cases are actually correct.

Among all models, the Random Forest model shows strong performance with high recall and precision, making it the most reliable model for fraud detection in this project.

8.4 Interpreting Model Behavior and System Output

The final system output provides a layered interpretation:

1. **Class (0/1):** The binary prediction for immediate action (block or allow).
2. **Fraud Probability (%):** The continuous value generated by `predict_proba`. This is crucial for operational decision-making, allowing institutions to set a dynamic threshold. For example, a transaction with a probability of 0.85 is clearly fraud, but a transaction at 0.55 may warrant further human review.
3. **Risk-level Interpretation:** The probability is mapped to risk levels:
 - Below 50% : Low Risk
 - 50% – 75% : Medium Risk
 - Above 75% : High Risk

This probability-based prediction system offers flexibility and transparency, which is a significant improvement over simple binary rule systems.

9. Conclusion

This project successfully developed and evaluated multiple machine learning classification models for the detection of fraudulent digital payment transactions, achieving the primary objective of building an intelligent and scalable detection system.

The core achievements include:

1. **Effective Feature Engineering:** The creation of novel features like `balance_diff_orig`, `is_full_transfer`, and `zero_dest` proved instrumental in extracting the latent, predictive signals

from the raw transaction data. These features, based on financial anomaly principles, significantly enhanced model performance.

2. **Robust Imbalance Handling:** The strategic application of the `class_weight='balanced'` technique successfully addressed the severe class imbalance, allowing the models to learn from the rare fraud cases effectively and shift the focus from misleading accuracy to the vital metrics of Recall and F1-score.
3. **Superior Model Selection:** The **Random Forest** ensemble classifier was identified as the top-performing model, achieving a high F1-score of 0.99, thereby offering the most reliable balance between detection rate (Recall) and false alarm rate (Precision).
4. **Actionable System Output:** The finalized system provides not just a binary output but a continuous fraud probability and risk-level interpretation, giving operational teams the granularity needed for timely and effective intervention.

In conclusion, this project demonstrates that data mining techniques, particularly ensemble methods coupled with domain-specific feature engineering, offer a reliable, adaptable, and significantly more sophisticated approach to fraud detection compared to traditional methods.

10. Future Work

The successful implementation of the Random Forest model establishes a strong foundation, but several avenues exist for future enhancement and system deployment:

1. **Real-time Deployment Pipeline:** The current models are trained offline. Future work should focus on integrating the trained model into a high-throughput, low-latency scoring engine (e.g., using technologies like Apache Flink or Kafka Streams) to enable real-time prediction and blocking of suspicious transactions immediately after they occur.
2. **Deep Learning Models:** Exploring advanced deep learning architectures, such as Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTMs), which are inherently capable of modeling sequential data (time series of transactions), could potentially uncover complex temporal fraud patterns that current static features may miss.
3. **Graph-based Fraud Detection:** Fraud often involves collusion and networks of malicious accounts. Future work could investigate Graph Neural Networks (GNNs) to model the transaction network itself. Nodes (accounts) and edges (transactions) in a graph can reveal community structures and centralized money laundering hubs that are difficult to detect with transaction-level analysis alone.
4. **System Scaling and Self-Learning:** Implementing a self-learning system that periodically retrains the model on new, labeled data would allow the detection system to constantly adapt to emerging fraud tactics. Further scaling the solution to handle Petabyte-scale transaction data would require distributed computing frameworks like Spark MLlib.

11. References

- [1] Scikit-learn Developers, “*Scikit-learn: Machine Learning in Python*,” Available: <https://scikit-learn.org>
- [2] Pandas Development Team, “*Pandas: Python Data Analysis Library*,” Available: <https://pandas.pydata.org>
- [3] Matplotlib Development Team, “*Matplotlib: Visualization Library for Python*,” Available: <https://matplotlib.org>
- [4] M. Waskom, “*Seaborn: Statistical Data Visualization*,” Available: <https://seaborn.pydata.org>
- [5] Kaggle, “*Fraud Detection Dataset and Related Projects*,” Available: <https://www.kaggle.com>
- [6] YouTube, “*Machine Learning and Fraud Detection Tutorials*,” Available: <https://www.youtube.com>

12. Appendix

This appendix provides select code snippets illustrating the critical steps of the project implementation.

A. Sample Code Snippet: Feature Engineering

The following Python code demonstrates the generation of the four critical engineered features using Pandas.

```
Python
import pandas as pd

# Feature Engineering

# Difference between sender's old and new balance
df['balance_diff_orig'] = df['oldbalanceOrg'] - df['newbalanceOrig']

# Difference between receiver's new and old balance
df['balance_diff_dest'] = df['newbalanceDest'] - df['oldbalanceDest']

# Indicator for full balance transfer
df['is_full_transfer'] = (df['amount'] == df['oldbalanceOrg']).astype(int)

# Indicator for zero balance destination account
df['zero_dest'] = (df['oldbalanceDest'] == 0).astype(int)
```

B. Sample Code Snippet: Random Forest Model Training

This snippet shows the instantiation and training of the Random Forest model, emphasizing the handling of class imbalance.

```
Python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# Split dataset
X = df.drop('isFraud', axis=1)
y = df['isFraud']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Initialize model with class imbalance handling
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)

# Train model
rf_model.fit(X_train, y_train)
```

C. Sample Code Snippet: Prediction with Probability and Risk Interpretation

The final output system requires generating both the class and the probability, which is then mapped to a risk level.

```
Python
# Predictions
rf_pred = rf_model.predict(X_test)

# Probability of fraud (class 1)
rf_probs = rf_model.predict_proba(X_test)[:, 1]

# Risk interpretation
def get_risk_level(prob):
    if prob >= 0.75:
```

```
        return "High Risk"
    elif prob >= 0.5:
        return "Medium Risk"
    else:
        return "Low Risk"

# Sample output
for i in range(5):
    print("Prediction:", rf_pred[i])
    print("Fraud Probability:", round(rf_probs[i]*100, 2), "%")
    print("Risk Level:", get_risk_level(rf_probs[i]))
    print()
```